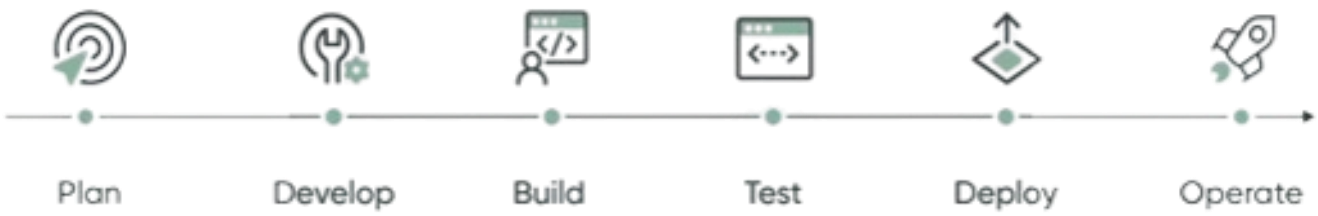
HOGESCHOOL ROTTERDAM / CMI

Software construction: process and tools



INFSCP01

Aantal studiepunten: 12
Modulebeheerder: S. de Redelijkheid

Modulebeschrijving

Modulenaam: Software construction: Process and tools

Modulecode: **INFSCP01**

Aantal studiepunten en studiebelastinguren:
Dit studieonderdeel levert de student 12 studiepunten op, hetgeen overeenkomt met een studielast van 336 uren.
De verdeling van deze 336 uren over de collegeweken is als volgt:

Begeleide lessen		
Kick-off bijwonen	1 x 2 uur	2 uur
Lessen inclusief workshops	16 weken x 2 x 4 uur	128 uur
Einddemonstratie geven en deelname	1x 4 uur	4 uur
Onbegeleid		
Werken aan het projectdeel	16 weken * 7 uur	112 uur
Zelfstudie en voorbereiding	16 weken * 5 uur	90 uur
Voorbereiding einddemonstratie	1 x 5 uur	5 uur
	Totaal	**336 uur**

Vereiste voorkennis: Kennis van jaar 1 (Basecamp, OODP en Project)

Werkvorm: Projectmatig onderwijs met workshops (teamwork)

Toetsing: Examinering op basis van ingeleverd projectwerk en het proces.

Leermiddelen: Géén verplichte literatuur

Draagt bij aan competentie: Manage & Control, Analyseren, Ontwerpen, Adviseren en Realiseren

Leerdoelen:

1. De student kan bestaande software analyseren, documenteren en valideren;
2. De student kan bestaande software analyseren, identificeren waar verbeteringen nodig zijn en refactoringtechnieken toepassen om de code te optimaliseren;
3. De student kan werken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden;
4. De student kan automatische tests schrijven en uitvoeren. Dit kunnen unit tests, integratietests en acceptatietests zijn;
5. De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie;
6. De student begrijpt hoe monitoring en logging in CI/CD-omgevingen werken en kan hier fouten uit opsporen om problemen op te kunnen lossen;
7. De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden zoals dependencies;
8. De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt.

Inhoud: Je leert werken in een projectgroep (maximaal 4 personen) om bestaande software te analyseren, documenteren en testen. Samen met je projectgroep ga je de bestaande software aanpassen (refactoren) en deze koppelen aan automatiseringstools binnen CI/CD, om deze vervolgens uit te breiden aan de hand van wensen van de Product Owner (client).

Opmerkingen: Aanwezigheid is verplicht.

Modulebeheerder: S. de Redelijkheid

Datum: 5 juni 2025

1. Algemene omschrijving

"The most powerful tool we have as developers is automation." — Scott Hanselman

1.1 Inleiding

Als software engineer ben je altijd op zoek naar manieren om dingen voor jou te laten werken. In deze cursus ga je in op automatiseringsslagen voor het ontwikkeltraject en de verbeteringen die je kan toepassen om een zo goed mogelijk stuk software af te leveren. Samen met je team ga je aan de slag met bestaande software, deze breng je in kaart, voorzie je van documentatie (al dan niet geautomatiseerd) en tests om te garanderen dat alles werkt zoals het zou moeten. Als je de basis hebt staan, ga je door middel van automatisering via CI/CD ervoor zorgen dat aanpassingen aan de software nog steeds werken. Als je samen met je team een werkende pipeline hebt opgezet, ga je beginnen met het uitbreiden of verbeteren van de software.

1.2 Relatie met andere onderwijseenheden

In het eerste jaar heb je leren programmeren in Python en C# waarbij verschillende concepten en moeilijkheidsgraden aan bod zijn gekomen. Dit jaar gaan we verder op de kennis die je hebt opgebouwd en deze uitbreiden met automatiseringsmogelijkheden die je helpen in je leertraject, stage en verdere carrière.

1.3 Leermiddelen

Learning GitHub Actions van O'Reilly Media er op komen [optioneel]
(https://www.oreilly.com/library/view/learning-github-actions/9781098131067/)

Learning UML 2.0 [optioneel]
(https://www.oreilly.com/library/view/learning-uml-20/0596009828/)

Efficient Linux at the Command Line [optioneel]
(https://www.oreilly.com/library/view/efficient-linux-at/9781098113391/)

Refactoring: Improving the Design of Existing Code [optioneel]
(https://www.oreilly.com/library/view/refactoring-improving-the/9780134757681/)

Effective Software Testing [optioneel]
(https://www.oreilly.com/library/view/effective-software-testing/9781633439931/)

Python Testing with pytest [optioneel]
(https://www.oreilly.com/library/view/python-testing-with/9781680509427/)

2. Programma

Je komt als ontwikkelaar te werken in een klein team (max 4 studenten), waarbij je samen aan bestaande (legacy) software in Python gaat werken op basis van de beschikbare casus. Gaandeweg dien je de software te analyseren, in kaart te brengen, te testen en vervolgens te refactoren. Tegelijkertijd ga je een CI/CD pipeline opzetten om gedurende het ontwikkeltraject altijd te blijven testen, loggen en monitoren. Als alles goed gaat, breidt je de applicatie uit met extra functionaliteit.

2.1 Technische requirements

Je begint het project in Python maar het staat je vrij om later te wisselen naar andere talen/frameworks/tools voor de uitwerking en verbetering van de applicatie. Zorg ervoor dat je dit bespreekt met je technisch docent om te zien of jouw keuze haalbaar is. Het is belangrijk om met de gekozen technologieën aan de criteria uit rubric (3.2) te voldoen. Als student heb je de mogelijkheid om zowel GitHub als GitLab te gebruiken, voor GitHub kan je als student gebruikmaken van het GitHub Student Developer Pack (https://github.com/edu/students), welke je vele extra's geeft en de mogelijkheid biedt om als Team te kunnen werken. Voor GitLab kan je de free version gebruiken of de GitLab server van de HRO gebruiken (vraag je technisch docent).

Voor beide oplossingen kan het voorkomen dat de hoeveelheid minuten voor de runners niet toereikend is, dan kan de school je voorzien van een Virtual Machine (VM) server die je kunt aanvragen via het volgende formulier: https://forms.office.com/e/Nc0DXbXTUy of bij de receptie op de begane grond van Wijnhaven 99, vlakbij de ingang. Hierop kan je dan je private runner installeren en koppelen om te gebruiken binnen GitHub Actions of GitLab.

2.2 Opleveren

Aan het einde van de cursus lever je een deel van het werk in als team en een deel op individuele basis, hieronder is opgesplitst wat ingeleverd dient te worden:

Team

- *Documentatie toegepaste projectmethodiek (scrum)*
- *Werkende CI/CD pipeline*
  - *Video werkende pipeline (max. 720p, max. 2 minuten)*
  - *YAML bestanden*
  - *Bash bestanden*
  - *Configuratie en dependency bestanden*
- *Aangepaste applicatie (code)*
- *Documentatie*
  - *Onderzoeksrapport*
  - *Technisch rapport*
  - *Testrapport*
  - *Swagger documentatie*
- *Presentaties*
  1. *Analyse bestaande software*
  2. *Pipeline*
  3. *Eindpresentatie*

Individueel

- Schriftelijk bewijs van de individuele bijdrage aan het proces (een logboek met taken en uren) Maak duidelijk dat je aan de voorwaarden voldoet (zie 3.1).
- Schriftelijk bewijs van de individuele bijdrage aan het product, de pipeline en de rapporten (codefragmenten, samenvatting van bijdrage/voltooide taken, git-logs). Let op: plagiaat wordt niet getolereerd;

2.3 Planning

Week	Uitvoering	Onderwerpen	Taken
1	Kick-off	Groepen vormen, casussen verdelen, opzetten omgevingen en samenwerkingscontract opstellen	
2	Workshop 1	REST, Web-API's, Requirements specifications, reverse-engineering	
3	Workshop 2	Opzetten van Git, aanmaken van branches en starten met het project op basis van issues.	
4	Workshop 3	Voordelen van testen, unit-, API-, integratie-, beveiliging & autorisatie-, prestatietesten	
5		Werken aan de tests voor de bestaande codebase en het controleren van de code coverage	Inleveren concept onderzoeksrapport
6	Workshop 4	Refactoren van de codebase en voortgang bespreken	Presentatie 1
7		Refactoren van de codebase en voortgang bespreken	Bespreken onderzoeksrapport
8	Workshop 5	Voordelen van CI/CD, kiezen van een platform (GitHub/Gitlab), aanmaken van eerste pipeline	
9		Het uitbreiden van de CI/CD pipelines en het opzetten van meerdere steps/jobs en voortgang bespreken	Inleveren concept testrapport
10	Workshop 6	Werken met bash en taken automatiseren en voortgang bespreken	Bespreken testrapport
11		Uitbreiden en valideren van de CI/CD pipelines en voortgang bespreken	
12		Refactoren van de codebase *(database toevoegen)* en voortgang bespreken	Presentatie 2
13	Workshop 7	UML en ERD maken voor het technisch rapport en voortgang bespreken	
14		Werken aan het product en de documentatie en voortgang bespreken	
15		Werken aan het product en de documentatie en voortgang bespreken	Inleveren concept technisch rapport
16		Werken aan het product en de documentatie en voortgang bespreken	Bespreken technisch rapport
17		Werken aan het product en de documentatie en het inleveren van alle documentatie en bijdragen	Alles inleveren
18	Demo day	Het tonen van het eindresultaat in een demo	Presentatie 3
19		*Werken aan herkansing (indien nodig)*	
20	Resit final	*Werken aan herkansing (indien nodig)*	Herkansing inleveren

2.4 Workshops

Workshop 1 - Requirements Specification
In deze workshop gaan we kijken naar het opstellen van requirements. Wat zijn goede requirements en waar moeten ze o.a. aan voldoen. Wat is REST en hoe werken API's, een korte introductie en hoe je requirements daarvoor opstelt. Aan de hand van voorbeelden ga je kijken naar hoe je o.a. requirements uit bestaande software zou kunnen ontleden.

Workshop 2 – Git
In deze workshop gaan we kijken naar het gebruik van Git en GitHub. Welke termen worden er gebruikt en waar dienen ze voor, wat is de normale gang van werken met Git en wat is het verschil met GitHub. Gedurende de gehele cursus is Git de levensader van CI/CD en nu kijken we naar waarvoor, waarom en hoe je het gebruikt.

Workshop 3 - Software Testing
In deze workshop gaan we kijken naar het testen van software. Wat test je wanneer, welke methoden zijn er om te testen en waar moet je rekening mee houden. Welke tools kan je gebruiken en hoe gaat dit straks helpen binnen onze CI/CD pipeline.

Workshop 4 – Refactoring
In deze workshop gaan we kijken naar refactoring, wat is het, waar gebruik je het voor en hoe pas je het toe? Hoe draagt refactoring bij aan het beter onderhoudbaar maken van je software en hoe kunnen we dit op een bestaand project toepassen?

Workshop 5 - CI/CD
In deze workshop gaan we kijken naar Continuous Integration (CI) en Continuous Development (CD), wat wordt er mee bedoeld, waar gebruiken we het voor en wat is het verschil tussen Continuous Development en Continuous Delivery. We kijken naar tools, instellingen en gaan een begin maken met de opzet.

Workshop 6 - Operating Systems + Bash
In deze workshop gaan we kijken naar het gebruik van Bash en de kracht binnen CI/CD ervan. Hoe verschillende Operating Systems van elkaar verschillen. Hoe je navigeer je tussen mappen, hoe je maak je bestanden aan, hoe kan je bestanden verplaatsen en verwijderen en hoe stellen we rechten in.

Workshop 7 – UML
In deze workshop gaan we dieper in op het gebruik en de kracht van UML als taal voor het visueel weergeven van software of veranderingen in software. Wij kijken onder andere naar de volgende onderdelen: Context diagram, ERD, Deployment diagram, Flowchart, State diagram, Swagger.

3. Toetsing en beoordeling

3.1 Beoordeling

Het op te leveren werk is een teamprestatie, waaraan iedereen individueel bijdraagt. Het cijfer wordt aan het team gegeven, maar er is een mogelijkheid om een verhoging (of verlaging) op individuele basis van maximaal +1 / -1 punt te krijgen op het groepscijfer (zie 3.4).
Individuele studenten die geen significante bijdrage leveren, kunnen echter worden uitgesloten van deze cursus en krijgen een No Go (een ND in Osiris). De docenten monitoren de bijdrage regelmatig en kunnen tijdens de cursus om verduidelijking vragen. Om een cijfer te krijgen voor deze cursus moet je aan de volgende voorwaarden voldoen:

Team:
- Je moet een projectmethodiek (scrum) toepassen;
- Je levert een werkende aangepaste applicatie op;
- Je levert een werkende CI/CD pipeline op;
- Je levert alle documentatie op (onderzoeksrapport, technischrapport + swagger, testrapport).

Individueel:
- Je voldoet aan de aanwezigheidseisen (zie einde paragraaf voor meer details);
- Je levert een significante bijdrage aan het proces:
- Je toont een betrokken/actieve houding binnen de cursus;
- Je bent een teamspeler;
- Je houdt je teamleden op de hoogte en houdt je aan de afspraken;
- Je levert een git-log aan van je contributie;
- Je levert een significante technische bijdrage aan het product (code, eisen, testen, etcetera);

Scenario's
Voldoe je tijdens de looptijd van de cursus niet aan één van bovenstaande criteria, dan ontvang je een officiële waarschuwing. Je krijgt dan de gelegenheid om de nodige wijzigingen aan te brengen om aan de ontbrekende criteria te voldoen, tenzij je niet aan de aanwezigheidseisen voldoet (niet herstelbaar).
- Als je slaagt binnen de eerste oplevering van de cursus, krijg je een eindcijfer;
- Lukt het niet om dit op tijd op te lossen, dan ontvang je bij de eerste poging een No Go/ND en zul je jezelf moeten bewijzen bij de herkansing;
- Als je er niet in slaagt de noodzakelijke wijzigingen aan te brengen en het niet mogelijk is om het probleem op te lossen, kun je de cursus niet opnieuw uitvoeren. Dit betekent dat je de cursus volgend studiejaar opnieuw moet volgen. De technisch docent en tutor beslissen of je bijdrage herstelbaar is of niet.

Aanwezigheid
Aanwezigheid is verplicht en de aanwezigheidseisen gelden zowel voor online als offline onderwijs.
De aanwezigheid wordt tijdens elke les gecontroleerd door de technisch docent en/of tutor. Om aan de aanwezigheidsplicht te voldoen, dient de student minimaal 80 procent van de lessen aanwezig te zijn, te laat (binnenkomst na starttijd rooster) telt als afwezig. Het is aan de decaan om te bepalen of je afwezigheid valt onder een 'bijzondere omstandigheid' (zoals langdurige ziekte of een begrafenis van een direct familielid), waarbij alleen in overleg met de docent(en) afgeweken kan worden van de aanwezigheidseis. Werk, een te plannen dokters- of tandartsbezoek, een griepje van een paar dagen of ov-problemen, etc. vallen niet onder een bijzondere omstandigheid. Stakingen in het ov zijn een uitzondering.

Voldoe je niet aan de aanwezigheidsplicht, dan ontvang je een No Go/ND. Je kunt de cursus niet opnieuw uitvoeren. Dit betekent dat je de cursus volgend studiejaar opnieuw moet doen.

Plagiaat
Onder plagiaat wordt verstaan: het overnemen van teksten, afbeeldingen of code zonder correcte bronvermelding. Hier vallen ook door kunstmatige intelligentie gegenereerde teksten, afbeeldingen of code onder, zoals ChatGPT. Je mag dus wel werk van anderen gebruiken in je eigen werk, als je maar duidelijk maakt dat het niet je eigen werk is en een correcte bronvermelding geeft. Plagiaat is een vorm van fraude.

3.2 Toetsmatrijs

	Leerdoelen	Niveau (Bloom)	Weging
L1	De student kan bestaande software analyseren, documenteren en valideren;	**Analyseren, Onthouden**	15%
L2	De student kan bestaande software analyseren, identificeren waar verbeteringen nodig zijn en refactoring technieken toepassen om de code te optimaliseren;	**Analyseren, Begrijpen**	15%
L3	De student kan werken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden;	**Toepassen**	10%
L4	De student kan automatische tests schrijven en uitvoeren. Dit kunnen unit tests, integratietests en acceptatietests zijn;	**Toepassen**	15%
L5	De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie;	**Analyseren, Toepassen**	15%
L6	De student begrijpt hoe monitoring en logging in CI/CD-omgevingen werken en kan hier fouten uit opsporen om problemen op te kunnen lossen;	**Analyseren, Begrijpen**	10%
L7	De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden zoals dependencies;	**Evalueren**	10%
L8	De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt.	**Creëren**	10%

3.3 Rubric

Elk leerdoel wordt aan de hand van een rubric nagekeken (zie bijlage beoordelingsformulier). De rubric bestaat uit 4 onderdelen (onvoldoende, voldoende, goed en uitstekend), met daaronder een mogelijk puntenaantal behorende bij die rubric (zie tabel – Voorbeeld rubric).

Tabel - Voorbeeld rubric

	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*Niet alle verplichte onderdelen zijn uitgewerkt of de uitwerking is van onvoldoende kwaliteit.*					*Alle verplichte onderdelen zijn uitgewerkt, maar er is ruimte voor verbetering.*		*Alle verplichte onderdelen zijn goed uitgewerkt en extra onderdelen zijn deels toegepast.*		*Alle onderdelen zijn uitstekend uitgewerkt waarbij er extra onderdelen en/of best-practices zijn toegepast.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										

3.4 Becijfering

De toetsing vindt plaats op basis van het opgeleverde product (pipeline, code, documents) en het proces van het project (project-log, git-logs, contributieverslag). Alle leeruitkomsten moeten minimaal een voldoende scoren om in aanmerking te komen voor een voldoende voor de hele cursus. Individuele contributie kan een ND opleveren (zie 3.4) of een individuele aanpassing op het groepscijfer tussen de -1 en +1 punt op basis van individuele contributie binnen de groep. Bij slechte prestatie kan er tot 1 punt van het groepscijfer afgetrokken worden, bij uitstekende prestaties kan er tot 1 punt boven het groepscijfer opgeteld worden (maximaal cijfer is 10).

Het eindcijfer wordt door zowel je tutor als je technisch docent bepaald. Voor elk leerdoel kan je maximaal 10 punten scoren. De volgende formule zal gebruikt worden voor de berekening van je eindscore:

$$Eindcijfer = min(\sum_{n=1}^{N} punten_leerdoel_n \cdot weging_leerdoel_n + individueel, 10)$$

3.5 Cesuur

- Als je een **NVL** (*niet voldaan)* hebt gekregen voor aanwezigheid,
  dan is herkansing dit schooljaar niet meer mogelijk.
- Als je een **NVL** (*niet voldaan)* hebt gekregen voor contributie en reparatie is niet meer mogelijk,
  dan is herkansing dit schooljaar niet meer mogelijk.
- Als je een **NVL** (*niet voldaan)* hebt gekregen voor contributie en reparatie is mogelijk,
  dan kan je gebruikmaken van de herkansing (zie 3.5).
- Als je **eindcijfer en/of één van de leerdoelen** niet toereikend is (lager dan 5,5),
  dan kan je gebruikmaken van de herkansing (zie 3.5).

Voorbeeld:
Je krijgt het volgende cijfer:
- **(Individueel/team) Voldaan** voor projectmethodiek, aanwezigheid en bijdrage aan het proces, de pipeline en het product (vereiste requirements)
- Uit de rubriek een eindcijfer = **6,7 (voldoende)**

Je krijgt het volgende cijfer:
- **(Individueel/team) Voldaan** voor projectmethodiek, aanwezigheid en bijdrage aan het proces, de pipeline en het product (vereiste requirements)
- Uit de rubriek een eindcijfer = **5,4 (onvoldoende)**, je moet de herkansing doen.

Je krijgt het volgende cijfer:
- **(Individuele) Niet voldaan** voor individuele bijdrage (niet te repareren), **ND (niet deelgenomen)**, je komt niet in aanmerking voor de herkansing.

Je krijgt het volgende cijfer:
- **(Individueel) Niet voldaan** voor aanwezigheid, **ND (niet deelgenomen**, je komt niet in aanmerking voor de herkansing.

3.6 Herkansing/ reparatie

Als het eindcijfer lager dan 5,5 is voor één of meer van de leerdoelen, of je hebt niet voldaan aan de minimumeisen, dan zal je gebruik moeten maken van de herkansing. Maximaal 2 leerdoelen mogen herkanst/gerepareerd worden, als er meer dan 2 leerdoelen onvoldoende zijn dan is herkansing niet mogelijk. Herkansing heeft een duur van 2 weken en dient voor het einde van week 20 ingeleverd te zijn.

Indien je de herkansing/reparatie niet haalt (lager dan 5,5) of als je een ND krijgt dan zal je deze cursus volgend schooljaar opnieuw moeten doen.

Bijlage I - Uitleg op te leveren producten

Presentaties

Presentatie 1:
Binnen deze presentatie neem je ons mee in de bevindingen welke jullie als groep hebben opgedaan na het analyseren van de bestaande software. Je stipt de belangrijkste kenmerken aan en beschrijft wat functionele en niet functionele requirements.

Presentatie 2:
Binnen deze presentatie neem je ons mee in de tests welke jullie als groep hebben opgesteld voor de bestaande applicatie. Jullie vertellen over de uit te voeren testsoorten die jullie gaan implementeren en waarom en welke tests daar o.a. onder vallen.

Presentatie 3:
In deze eindpresentatie nemen jullie als groep ons mee door het project. Je vertelt kort over jullie analyse, de requirements die daaruit kwamen en de validatie ervan. Welke tests er opgezet zijn en waarom deze gekozen zijn. Hoe de refactoring heeft plaatsgevonden en wat het resultaat ervan is en eventueel welke uitbreidingen er gemaakt zijn. Ook laat je de CI/CD pipeline zien en welke verbeteringen op documentatieniveau er gemaakt zijn om de uitbreidbaarheid van de software te vergroten.

Rapporten

Onderzoeksrapport:
Het onderzoeksrapport geeft een duidelijk inzicht in hoe jullie als groep te werk zijn gegaan bij het analyseren van de bestaande software en welke requirements daaruit onttrokken zijn. Hiernaast geven jullie aan hoe jullie deze requirements hebben gevalideerd en welke resultaten dit opgeleverd heeft. Tot slot sluit je af met de conclusie en eventuele aanbevelingen. Dit alles verwerk je netjes in een rapportage.

Indeling kan zijn:
- Voorblad
- Inleiding
- Methodologie
- Analyse bestaande software
- Requirements
- Validatie
- Resultaten en discussie
- Conclusie en aanbevelingen
- Bijlagen

Testrapport:
Het testrapport geeft een duidelijk inzicht in hoe jullie als groep de software getest hebben, welke methodes en technieken ervoor gebruikt zijn en wat de uitkomsten hiervan waren.

Indeling kan zijn:
- Voorblad
- Inleiding
- Teststrategie en -aanpak
- Testgevallen en resultaten
- Defecten en problemen
- Analyse van resultaten
- Conclusie en aanbevelingen
- Bijlagen

Technisch rapport:
Het technisch rapport geeft een gedetailleerde weergave van het technische onderdeel van de applicatie. Doormiddel van het technisch rapport zou een ander in staat moeten zijn te starten en/of verder te gaan met de ontwikkeling/uitbreiding van de applicatie.

Indeling kan zijn:
- Voorblad
- Inleiding
- Methode
  - Algoritmes
  - Ontwerp
  - Implementatie
  - Procedure
  - Data-analyse
- CI/CD pipeline
- Resultaten
- Discussie
- Conclusie en aanbevelingen
- Bijlagen

Koppelmatrix
In onderstaande tabel is terug te vinden welke producten gekoppeld zijn aan welke leerdoelen.

Product(en)	**Leerdoelen**
Projectdocumentatie (scrum)	L3, L8
Werkende CI/CD pipeline	L5, L6, L7
Aangepaste applicatie	L1, L2, L3, L4, L5, L6, L7
Onderzoeksrapport	L1, L7
Technisch rapport	L2, L3, L7
Testrapport	L2, L4, L5
Swagger documentatie	L1, L2
Presentaties	L1, L2, L4, L5, L7
Bewijs individuele bijdrage	L1, L2, L3, L4, L5, L6, L7, L8

Bijlage || - Beoordelingsformulier

L1	De student kan bestaande software analyseren, documenteren en valideren									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft de software niet geanalyseerd, gedocumenteerd of gevalideerd.* *Er zijn ernstige tekortkomingen in de uitvoering van de opdracht.*					*De student heeft de software gedeeltelijk geanalyseerd, gedocumenteerd en gevalideerd.* *Er zijn enkele verbeterpunten, maar de basisvereisten zijn voldaan.*		*De student heeft de software grondig geanalyseerd, gedocumenteerd en gevalideerd.* *Er zijn geen grote fouten en de opdracht is goed uitgevoerd.*		*De student heeft de software uitmuntend geanalyseerd, gedocumenteerd en gevalideerd.* *Het werk is van hoge kwaliteit en toont diepgaand begrip.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L2	**De student kan bestaande software analyseren, identificeren waar verbeteringen nodig zijn en refactoring technieken toepassen om de code te optimaliseren**									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft de bestaande code niet geanalyseerd of verbeterpunten geïdentificeerd.* *Er zijn geen refactoring technieken toegepast om de code te optimaliseren.*					*De student heeft de bestaande code gedeeltelijk geanalyseerd en enkele verbeterpunten geïdentificeerd.* *Er zijn enkele refactoring technieken toegepast om de code te optimaliseren, maar niet op alle relevante gebieden.*		*De student heeft de bestaande code grondig geanalyseerd en duidelijke verbeterpunten geïdentificeerd.* *Er zijn effectieve refactoring technieken toegepast om de code te optimaliseren op meerdere relevante gebieden.*		*De student heeft de bestaande code grondig geanalyseerd en duidelijke verbeterpunten geïdentificeerd.* *Er zijn effectieve refactoring technieken toegepast om de code te optimaliseren op meerdere relevante gebieden.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										

<table>
<tr><td>L3</td><td colspan="10">De student kan werken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden</td></tr>
<tr><td colspan="11"></td></tr>
<tr><td></td><td colspan="5">Onvoldoende</td><td colspan="2">Voldoende</td><td colspan="2">Goed</td><td>Uitstekend</td></tr>
<tr><td>Omschrijving</td><td colspan="5">De student heeft geen ervaring met Git aangetoond.

Er zijn geen branches aangemaakt, samengevoegd of beheert.</td><td colspan="2">De student heeft basiskennis van Git aangetoond.

Er zijn eenvoudige branches aangemaakt en samengevoegd en branches worden beheert met minimale fouten.</td><td colspan="2">De student heeft een goede kennis van Git aangetoond.

Er zijn complexere branches aangemaakt en samengevoegd en branches worden efficiënt beheert met eventueel begrip van Git-flows.</td><td>De student heeft diepgaande kennis van Git aangetoond.

Er zijn geavanceerde Git-technieken toegepast en branches worden foutloos beheert en geoptimaliseerd via workflows.</td></tr>
<tr><td></td><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td><td>6</td><td>7</td><td>8</td><td>9</td><td>10</td></tr>
<tr><td>Argumentatie</td><td colspan="10"></td></tr>
</table>



L4	De student kan automatische tests schrijven en uitvoeren. Dit kunnen unit tests, integratietests en acceptatietests zijn									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft geen ervaring met automatische tests.* *Er zijn geen unit tests, integratietests of acceptatietests geschreven.*					*De student heeft basiskennis van automatische tests.* *Er zijn eenvoudige unit tests geschreven en uitgevoerd en er is enig begrip van het belang van testdekking.*		*De student heeft een goede kennis van automatische tests.* *Er zijn complexere unit tests en integratietests geschreven en uitgevoerd en er is begrip van het testproces, waarbij testresultaten geanalyseerd worden voor verbeteringen.*		*De student heeft diepgaande kennis van automatische tests.* *Er zijn geavanceerde unit tests, integratietests en acceptatietests geschreven en uitgevoerd en teststrategieën zijn geoptimaliseerd aan de hand van best-practices.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L5	**De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie**									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft geen ervaring met CI/CD-pipelines.* *Er is geen basisconfiguratie voor GitHub Actions of GitLab CI/CD opgezet.*					*De student heeft basiskennis van CI/CD-pipelines.* *Er is een eenvoudige pipeline geconfigureerd met stappen voor build en testen.*		*De student heeft een goede kennis van CI/CD-pipelines.* *Er is een complexere pipeline geconfigureerd met meerdere stappen en er zijn optimalisatie van de pipeline doorgevoerd.*		*De student heeft diepgaande kennis van CI/CD.* *Er is een geavanceerde pipeline geconfigureerd met integratie van externe services. En de pipeline is geoptimaliseerd voor snelheid, betrouwbaarheid en veiligheid.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L6	**De student begrijpt hoe monitoring en logging in CI/CD-omgevingen werken en kan hier fouten uit opsporen om problemen op te kunnen lossen**									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft geen begrip van monitoring en logging in CI/CD-omgevingen.* *Er zijn geen fouten opgespoord of problemen opgelost met betrekking tot monitoring en logging.*					*De student heeft een basisbegrip van monitoring en logging in CI/CD-omgevingen.* *Er zijn eenvoudige fouten opgespoord en enkele problemen zijn met behulp van monitoring en logging opgelost.*		*De student heeft een goede kennis van monitoring en logging in CI/CD-omgevingen.* *Fouten worden deels automatisch opgespoord en problemen zijn met behulp van monitoring en logging opgelost.*		*De student heeft diepgaande kennis van monitoring en logging in CI/CD-omgevingen.* *Fouten worden automatisch opgespoord en problemen zijn met behulp van monitoring en logging opgelost en verbeterd.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L7	**De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden zoals dependencies**									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft geen begrip van beveiligingsaspecten.* *Credentials zijn niet veilig opgeslagen, er is geen encryptie teogepast of controle van kwetsbaarheden zoals dependencies is niet uitgevoerd.*					*De student heeft een basisbegrip van beveiligingsprincipes.* *Credentials zijn veilig opgeslagen, er wordt gebruik gemaakt van encryptie en er wordt gecontroleerd op eventuele dependency kwetsbaarheden.*		*De student heeft diepgaand begrip van hoe beveiliging werkt.* *Er worden effectieve maatregelen genomen tegen mogelijke kwetsbaarheden zoals dependencies en/of de eigen code.*		*De student heeft een uitgebreid begrip van beveiligingspraktijken.* *Beveiligingsmaatregelen worden proactief geïmplementeerd en best practices worden gevolgd.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										



L8	**De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt.**									
	Onvoldoende					Voldoende		Goed		Uitstekend
Omschrijving	*De student heeft moeite met effectieve communicatie binnen het team.* *Er is geen constructieve feedback gegeven of ontvangen en er zijn geen of slechts enkele codereviews uitgevoerd.*					*De student communiceert redelijk goed binnen het team.* *Er kan basisfeedback gegeven en ontvangen worden en de basisprincipes zijn toegepast op de codereviews.*		*De student is een effectieve communicator.* *Er is onderling constructieve feedback gegeven en ontvangen. En codereviews zijn toegepast waarbij suggesties en verbeteringen zichtbaar zijn uitgevoerd.*		*De student is een uitstekende teamspeler.* *Er is helder en respectvol gecommuniceerd en er is een positieve teamdynamiek. Codereviews zijn adequaat en correct uitgevoerd met zichtbare suggesties en verbeteringen.*
	1	2	3	4	5	6	7	8	9	10
Argumentatie										

Software Construction: Processing and Tools

Klas		Team		Poging	1 / 2	
Leerdoelen					**Weging**	**Punten**
L1	De student kan bestaande software analyseren, documenteren en valideren				15%	
L2	De student kan bestaande software analyseren, identificeren waar verbeteringen nodig zijn en refactoring technieken toepassen om de code te optimaliseren				15%	
L3	De student kan werken met Git als versiebeheersysteem. Er kunnen branches gemaakt, samengevoegd en beheerd worden				10%	
L4	De student kan automatische tests schrijven en uitvoeren. Dit kunnen unit tests, integratietests en acceptatietests zijn				15%	
L5	De student kan een CI/CD pipeline configureren binnen GitHub Actions of GitLab CI/CD met stappen voor build, testen en implementatie				15%	
L6	De student begrijpt hoe monitoring en logging in CI/CD-omgevingen werken en kan hier fouten uit opsporen om problemen op te kunnen lossen				10%	
L7	De student is zich bewust van beveiligingsaspecten, zoals het veilig opslaan van credentials, encryptie en het controleren van kwetsbaarheden dependencies				10%	
L8	De student kan effectief samenwerken in een teamomgeving, waarbij code wordt beoordeeld, feedback wordt gegeven en communicatie plaatsvindt.				10%	
Vereisten						
Studentnaam Studentnummer	{*naam*} {nummer}	{*naam*} {nummer}	{*naam*} {nummer}	{*naam*} {nummer}		
Aanwezigheid	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL		
Contributie aan het proces	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL		
Contributie aan het product	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL		
Contributie aan de pipeline	VLD / NVL	VLD / NVL	VLD / NVL	VLD / NVL		
Correctie voor inspanning	-1 / 0 / +1	-1 / 0 / +1	-1 / 0 / +1	-1 / 0 / +1		
Resultaat						
Eindcijfer						

Datum:

Technisch docent
Naam:

Handtekening:

Tutor
Naam:

Handtekening:

Extra onderbouwing:

Omschrijving reparatie: